

C-Real stddev profiling protokol

1. Detaily testu

Host: Ryzen 7 5800H, 32GB RAM, Windows 11 + WSL 2.0 Ubuntu 24.04.4 LTS, Dart SDK version: 3.11.5 (stable)
Commit hash: d20457ee4eddca51be783dfcf3ef8085d81682df
Příkaz: dart run --observe benchmark/profiling.dart
(CWD = src/packages/stddev, stejné závislosti jako pro build)

2. Výstup příkazu

```
The Dart VM service is listening on http://127.0.0.1:8181/qtoOyMLpFn4=/
The Dart DevTools debugger and profiler is available at:
http://127.0.0.1:8181/qtoOyMLpFn4=/devtools/?uri=ws://127.0.0.1:8181/qtoOyMLpFn4=/ws
Running in-process benchmarks using randomly generated doubles from the range <-10000; 10000>...

[profiler] CPU sample buffer cleared - recording started.

StdDev_____10(RunTime): 966.752 us.
StdDev____100(RunTime): 10541.382198952879 us.
StdDev___1000(RunTime): 105934.25 us.
StdDev__10000(RunTime): 1381416.5 us.
StdDev_100000(RunTime): 12938932.5 us.

[profiler] Profile saved → benchmark/profile.json
[profiler] To view: open Dart DevTools → CPU Profiler → "Load from file".
vm-service: isolate(4633370711040455)  'main' has no debugger attached and is paused at exit.
Connect to the Dart VM service at http://127.0.0.1:8181/qtoOyMLpFn4=/ to debug.
```

A přiložený soubor vystup.json (lze prohlížet v webovém GUI spuštěném pomocí příkazu dart devtools, záložka CPU Profiler)

3. Metodika/odůvodnění specifik

Důvod, proč byl test spuštěn v prostředí Linux a ne na Windows je omezení Windows kernelu naplánovat přerušení s maximální přesností 15ms, což omezí sampling rate profileru Dartu na 64Hz. Teoreticky vše bude fungovat, nicméně to značně omezí přesnost a spolehlivost výsledků.

Kód není spouštěn zcela tak, jak by byl spouštěn za normálního provozu v terminálu (stddev < input). Celý proces vypadá takto:

1. Příkaz dart run spustí v interpretoru soubor profiling.dart

2. Připraví se profilovací nástroje Dartu
3. Vygenerují se datasety náhodných čísel o požadované velikosti
4. Zapne se Dart profiler
5. Provedou se benchmarky (oproti běžnému provozu se liší v tom, že stddev.dart běží v interpretaci a vstup nedostává na stdin, ale jako String, který si kód nicméně stále musí parsnout na jednotlivé čísla)
6. Ukončí se Dart profiler, výstup se uloží do profile.json a ve webovém rozhraní lze prohlížet výsledky.

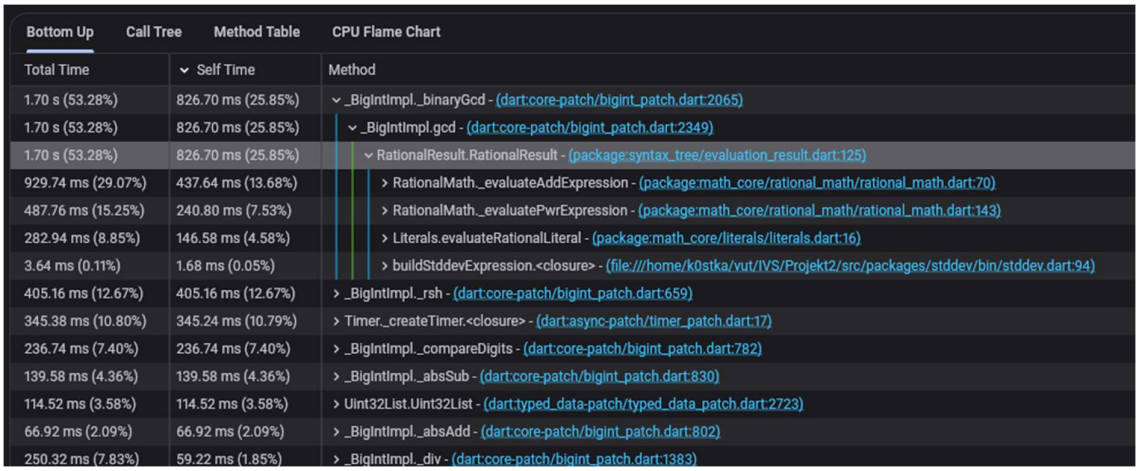
Důvod, proč je kód spouštěn takto, a ne běžným způsobem je ten, že v normálním způsobu jsem narazil na obtíže připojit Dart profiler dříve, než celý výpočet proběhl. (Běžně profiling v Dartu vypadá tak, že DartVM (název runtime – konceptuálně podobné C#) na začátku spustí webserver s GUI debuggeru/profileru apod., člověk se pak musí připojit a manuálně zapnout záznam profileru). Zkoušel jsem i variantu, kdy jsem na začátek kódu dal 20s sleep, nicméně pak jsem narazil na paradoxní problém, že jsem tímto zastavil i webserver s GUI profileru a nemohl jsem tím pádem spustit profiling dříve, než už bylo pozdě.
Za potencionální nevýhody tohoto přístupu by se dalo považovat potencionální rozdíl relativní délky průběhu interpretovaných vs kompilovaných funkcí a skutečnost, že v profilingu není obsaženo čtení ze stdin. První nepovažuji za pravděpodobné vzhledem k detailům interní implementace DartVM. U druhého z dodatečného testování vím, že v Dartu tvoří relativně velikou část (u 100k datasetu z ~1.1s celkové doby výpočtu tvořilo čtení ze stdin cca 350ms), nicméně jelikož toto je nevyhnutelná část, tak nepovažuji její absenci v flamechartu za kritickou.

4. Závěr

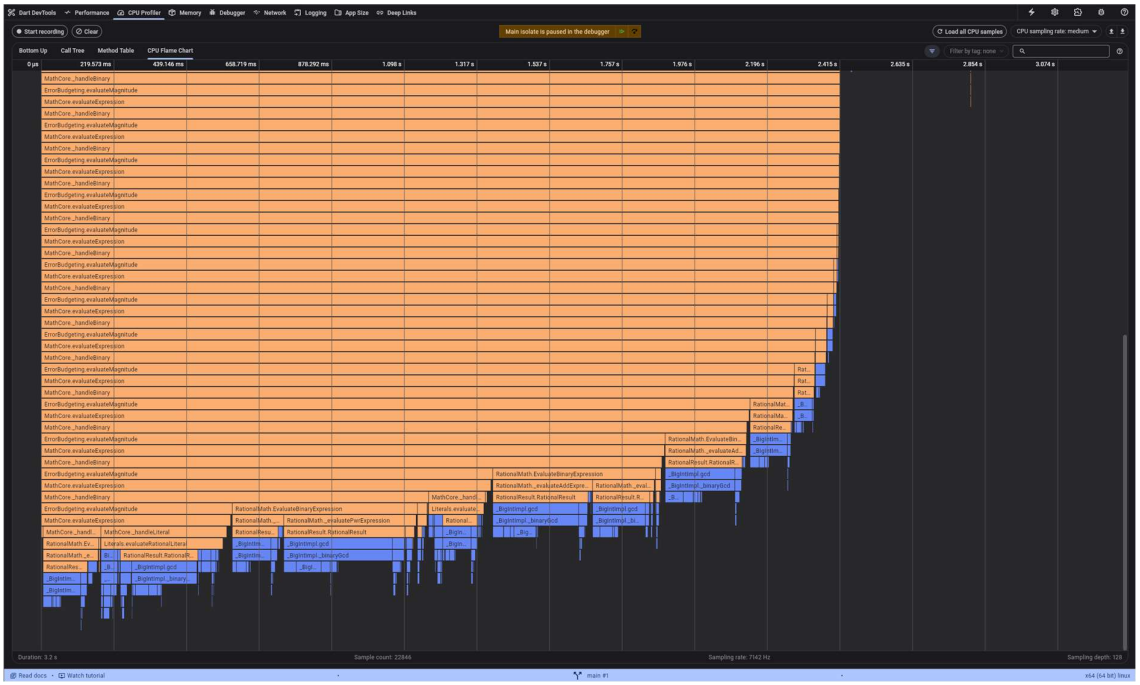
Na základě stejného nárůstu délky výpočtu a velikosti datasetu lze s radostí konstatovat, že kód má s největší pravděpodobností složitost blížící se O(N), což je vzhledem k povaze testovacího případu žádoucí.

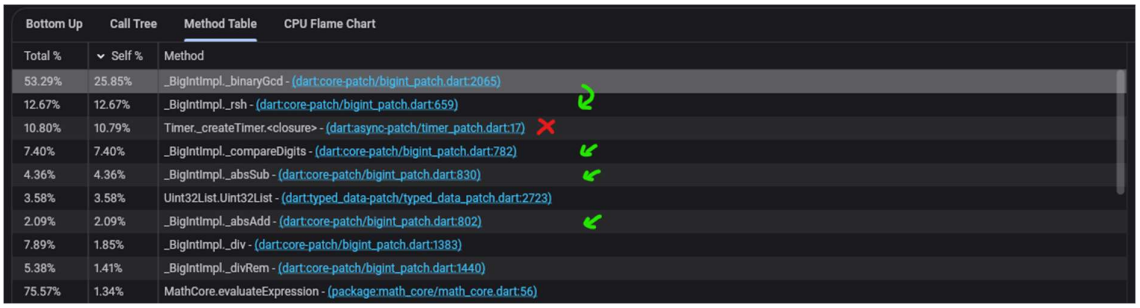
Konkrétně pak z hlediska výkonu je největší problém interní funkce Dartu: _BigIntImpl._binaryGcd, která je primárně volaná (přes funkci _BigIntImpl.gcd) z konstruktoru třídy RationalResult. Jedná se o funkci, která spočítá největšího společného dělitele dvou BigIntů (znamenková celá čísla s dynamickou délkou), které jsou v MathCore

(matematická knihovna) interně používány kdykoliv je pracováno s čísly pro zajištění možnosti nekonečné přesnosti. Třída RationalResult je pak používána jako uložiště mezivýsledku výpočtu daného podstromu v případě, že je knihovna výpočet schopna provést v oboru $\mathbb{Q}$. V ten okamžik RationalResult číslo ukládá jako podíl 2 BigIntů numerator/denominator. Tento zlomek je z kontraktu vždy v základním tvaru. K zajištění této skutečnosti jsou oba numerator a denominator v konstruktoru vyděleny jejich největším společným dělitelem, jelikož na vstupní hodnoty konstruktoru tento požadavek není.



(Podpoložky v rozbalovacím seznamu jsou funkce, které danou funkci zavolaly)


(Modře jsou ve flame chartu interní funkce Dartu)



(Funkce s červeným křížkem je šum od benchmarking harness, funkce se zelenou šipkou jsou volány hlavně z _BigIntImpl._binaryGcf)

5. Řešení

Bohužel řešení není tak jednoduché, jak se možná na první pohled může zdát. Prostým zrušením slibu, že RacionalResult ukládá zlomek v základním tvaru a ukládáním zlomku v tom tvaru, který vznikne jako výsledek výpočtů paradoxně výkon ještě klesne. Důvod je ten, že procesory na HW úrovni z logických důvodů nemají akceleraci pro BigInt (nemají nekonečné registry). Náročnost aritmetických operací BigIntů tedy roste s jejich délkou (interně jsou implementovány zhruba jako list int32). Bez opakovaného zmenšování hodnoty čitatele a jmenovatele pak tyto čísla velmi rychle narostou (hlavní problém jsou operace pro součet a přirozenou mocninu racionálních čísel, které jsou implementovány následovně: a/b + c/d = (ad + bc) / bd a (a/b)^n = a^n/b^n). Bohužel nepomůže ani vynutit výpočet v oboru reálných čísel, který teoreticky nemusí řešit normalizaci zlomků a teoreticky by nedošlo ani k ohromnému nárůstu požadavků na velikost error bound u listových prvků AST (abstract syntax tree), jelikož sumy prvků v rovnici pro standardní odchylku jsou skládány tak, aby tvořily balancovaný binární strom = minimalizují maximální hloubku. Problém je však ten, že matematická knihovna je celá prosetá optimalizacemi, které předpokládají možnost/nemožnost provádět určité operace v oboru racionálních čísel, takže by se jednalo o poměrně veliký zásah.

Nejlepším řešením by pravděpodobně byla určitá forma heuristiky, která by normalizovala zlomky pouze kdyby absolutní hodnota čitatele nebo jmenovatele přesáhla stanovený threshold.